॥ श्रीगणेशाय नमः ॥

Miscellaneous Topics

OLAP indexing → Bitmap, Join

Bitmap indexing

- A bitmap index represents data as a series of bitmaps.
- Each bitmap corresponds to a distinct value in a dimension.
- Each bit in a bitmap represents either the presence or absence of that value (1/0) for a particular row in the data.

E.g. does a resident in a city have internet access?

Identifier	HasInternet	Bitmaps	
		Y	N
1	Yes	1	0
2	No	0	1
3	No	0	1
4	Unspecified	0	0
5	Yes	1	0

Transaction ID	Product	Region	Time	Sales Amount
1	A	North	2024-08-01	100
2	B	South	2024-08-01	150
3	A	North	2024-08-02	200
4	C	West	2024-08-02	120
5	B	North	2024-08-03	300
6	A	South	2024-08-03	180

Step 1: Identify the unique values in the column.

{North, South, West}

Step 2: Create bitmaps for the unique values.

Tid	Product	North	South	West
1		1	0	0
2		0	1	0
3		1	0	0
4		0	0	1
5		1	0	0
6		0	1	0

Step 3: Create an index table.

Region	Bitmap
N	101010
S	010001
W	000100

Why this helps?

consider a query:
Find all the transac^n^s where region = "North" and time = "207~".
we'll just perform bitwise AND operat^n^s to get desired results.

Also it is really space efficient.

cons:

- Inefficient for high-cardinality data attributes.
  high cardinality data on a specific attribute.
  # of bitmaps for a single attribute would be too much.
  that'd be inefficient.
- updating bitmap indices is also not easy.

Join indexing

- Joins are widely used operations between factual and dimension or multiple dimension tables.
- Join indexing is used to precompute and store the indentifier pairs of the join results.

E.g. Consider the 2 dimensions: location and time, and the sales factual table.

Dimension table: locations

Location _id	City	Province	...
L08	Vancouver	BC	...
L12	Victory	BC	...

Fact table: sales

TID	Location_id	Item_id	...	Amount
T57	L08	I8569	...	849.98
T238	L12	I7253	...	1099.95
T459	L12	I8569	...	799.95
T884	L08	I3051	...	2799.99

Dimension table: items

Item _id	Model	category	...
I7253	iPhone	smartphone	...
I8569	Galaxy	smartphone	...
I9051	iMac	Desktop	...

Join index table for location_sales:

JD	location_id	TID
1	L08	T57
2	L12	T238
3	L12	T459
4	L08	T884

mostly warehouses use column storages

Column-based databases

- storage that'll lead to efficient OLAP queries (mostly read).
- Generally OLAP queries compute the aggregations of all the records on a small # of attributes.

TransactionID	Date	ProductID	CustomerID	Amount
1	2024-08-01	1001	501	250
2	2024-08-02	1002	502	300
3	2024-08-03	1003	503	450
4	2024-08-04	1004	504	100

row storage →

Col Storage :-
1. TID = {1, 2, 3, 4}
2. Date = {"2024-08-01", 2024-08-02", 2024-08-03", "2024-08-04"}
3. PID = {1001, 1002, 1003, 1004}
4. CID = {501, 502, 503, 504}
5. Amount = {250, 300, 450, 100}

query: SELECT SUM(amount) FROM sales
WHERE pid = 1003
AND date = '2024-08-02"

first we form a bitmap from pid = 1003
→ 0010
bitmap from date
→ 0100
Take AND

```
0010
0100
----
0000
```

← this would give apt. records.

Data cube materialization

(precomputing and storing various combinations across diff. dimensions.

Say data cube has n dimensions → 2^n cuboids.

(these may be reqd.)

So among these, we choose frequently needed ones and pre-compute values.

- we can have 2^n cuboids for a data cube having n dimensions.

- Full materialization : compute and store all combinations of n dimensions. (2^n cuboids)
- Partial materialization : compute subset of data cuboid values. (based on anticipation) — we generally are aware of what queries we'll fire.
  ✓ mostly used
  drawback:
  - All combinations may not be precomputed, but that's fine.
- No materialization : no pre-computation. Do on queries. (on-the-fly computation)